



AUDIT REPORT

September 2026

For

 **Monochrome**

Table of Content

Executive Summary	04
Number of Security Issues per Severity	06
Summary of Issues	07
Checked Vulnerabilities	08
Techniques and Methods	10
Types of Severity	12
Types of Issues	13
Severity Matrix	14
Impact	14
Likelihood	14
■ Informational Severity Issues	15
1. EIP-2612 Permit Does Not Support ERC-1271 Contract Wallets	15
2. Tokens Sent To The Token Contract Are Permanently Locked	16
3. Standard ERC-20 Allowance-Replacement Race / Permit Front-Running	17
4. Irreversible Burns And Burn-Within-Allowance	18
5. Full Genesis Supply Concentrated In A Single Treasury	19
Functional Tests	20
Threat Model	21
1. External Dependencies & Trust Boundaries	21
2. Entry & Exit Points Analysis (Function-Level)	23

2.1Function Entry / Exit Table	23
3. Asset Flow Mapping (Critical Contracts)	33
3.1Asset-Holding Contracts	33
3.2 Asset Entry & Exit Functions	33
Automated Tests	35
Closing Summary & Disclaimer	36

Executive Summary

Project Name	Monochrome (MCR)
Protocol Type	ERC20
Project URL	https://www.monochrome.exchange/en
Overview	Monochrome is a standard ERC-20 utility token with a fixed initial supply of 210,000,000 MCR (18 decimals). The entire supply is minted exactly once in the constructor to a single <code>initialHolder</code> . The contract integrates <code>ERC20Burnable</code> (holders, or approved spenders within allowance, can permanently reduce supply) and <code>ERC20Permit</code> (EIP-2612 gasless approvals). There is no owner, no role or admin function, no mint after construction, no upgradeability, no pausability, no blacklist, no rescue function and no fee-on-transfer logic.
Audit Scope	The scope of this Audit was to analyze the MCR Token Smart Contracts for quality, security, and correctness.
Source Code Link	https://github.com/monochrome-exchange/mcr-token/blob/main/src/Monochrome.sol
Contracts in Scope	<code>src/Monochrome.sol</code>
Branch	Main
Commit Hash	98a7bdc229f3c52f68b86cb51c23887e61fb8635

Language	Solidity
Blockchain	EVM
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	28th September 2026
Updated Code Received	28th September 2026
Review 2	29th September 2026
Fixed In	b5a5b64a92c4c38f21409aba3c7b9332428819c6

✔ Verify the Authenticity of Report on QuillAudits Leaderboard:
<https://www.quillaudits.com/leaderboard>

Number of Issues per Severity



Critical	0 (0%)
High	0 (0%)
Medium	0 (0%)
Low	0 (0%)
Informational	5 (100%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	0	5
	Partially Resolved	0	0	0	0	0
	Resolved	0	0	0	0	0

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	EIP-2612 Permit Does Not Support ERC-1271 Contract Wallets	Informational	Acknowledged
2	Tokens Sent To The Token Contract Are Permanently Locked	Informational	Acknowledged
3	Standard ERC-20 Allowance-Replacement Race / Permit Front-Running	Informational	Acknowledged
4	Irreversible Burns And Burn-Within-Allowance	Informational	Acknowledged
5	Full Genesis Supply Concentrated In A Single Treasury	Informational	Acknowledged

Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls

✓ Missing Zero Address Validation

✓ Private modifier

✓ Revert/require functions

✓ Multiple Sends

✓ Using suicide

✓ Using delegatecall

✓ Upgradeable safety

✓ Style guide violation

✓ Using throw

✓ Unsafe type inference

✓ Using inline assembly

✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments, match logic and expected behavior.
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper.
- Implementation of ERC standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity static analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open Security vulnerabilities identified that must be resolved and are currently unresolved.	Resolved These are the issues identified in the initial audit and have been successfully fixed.
Acknowledged Vulnerabilities which have been acknowledged but are yet to be resolved.	Partially Resolved Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		 High	 Medium	 Low
Likelihood	 High	Critical	High	Medium
	 Medium	High	Medium	Low
	 Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Informational Severity Issues

1. EIP-2612 Permit Does Not Support ERC-1271 Contract Wallets

Acknowledged

Path

`src/Monochrome.sol`

Function Name

`permit(address,address,uint256,uint256,uint8,bytes32,bytes32) (inherited ERC20Permit)`

Description

ERC20Permit validates the permit signature with ECDSA.recover, i.e. it requires a raw secp256k1 EOA signature over the EIP-712 digest. It does not consult IERC1271.isValidSignature. The genesis supply is minted to a 3-of-4 Safe, and a Safe (or any contract wallet) cannot produce a valid ECDSA permit signature, so permit cannot be used by the entity that holds 100% of the supply. The repository's own deployment guide documents this limitation.

Impact

Functional limitation only. Contract wallets must approve and transfer through ordinary on-chain transactions. There is no path by which this enables unauthorized allowance, minting, or theft; a Safe-signed EIP-712 message simply fails signature recovery and reverts with ERC2612InvalidSigner.

Likelihood

Certain for any contract-wallet holder; not attacker-dependent.

Recommendation

This is expected EIP-2612 behavior and is acceptable for the stated design ("EIP-2612 Permit is supported"). If gasless approvals are required for the treasury Safe, document that Safe operations are transaction-based, or use a separate EIP-2612-aware module. Optionally consider EIP-1271-aware permit in a future revision.

Status

Acknowledged (documented in docs/deployment.md and docs/holder-safety.md)

Informational Severity Issues

2. Tokens Sent To The Token Contract Are Permanently Locked

Acknowledged

Path

`src/Monochrome.sol`

Function Name

`transfer / transferFrom`

Description

The token contract has no function that can move tokens it holds. Any MCR transferred to the token contract address by mistake becomes permanently unrecoverable, and no admin exists to rescue it. Likewise, MCR sent to a mistyped address (including 0xdead) is not burned and cannot be reclaimed.

Impact

Only the sender of the erroneous transfer is affected; the loss is irreversible. Core supply accounting is unaffected; a transfer to the contract is a normal balance change and does not mint or burn, so it does not threaten other holders.

Likelihood

Low; requires an accidental or mistaken transfer to the contract address.

Recommendation

If desired, document that users must never send MCR to the token contract, and rely on wallet address checks / ENS to avoid mistyped destinations.

Status

Acknowledged (README Security warning and docs/holder-safety.md)

Informational Severity Issues

3. Standard ERC-20 Allowance-Replacement Race / Permit Front-Running

Acknowledged

Path

`src/Monochrome.sol`

Function Name

`approve(address,uint256) and permit(...)`

Description

approve overwrites the current allowance rather than increasing it (OpenZeppelin v5 removed increaseAllowance/decreaseAllowance). If an owner changes an allowance from an old non-zero value to a new non-zero value, an approved spender can front-run the change and spend the old allowance before the new value is mined, obtaining both. An unsubmitted, unexpired EIP-2612 permit is also not invalidated by an ordinary approve(0); it remains executable until its nonce is used or its deadline passes.

Impact

The classic ERC-20 allowance race. It can cause a spender to receive more than the owner intended in a specific front-running scenario. It does not affect totalSupply, other holders, or the ability to mint/burn beyond authorization.

Likelihood

Low; requires an owner to change a non-zero allowance to another non-zero allowance while a malicious spender can and does front-run the transaction.

Recommendation

Use exact-amount approvals (or permit with an exact value) rather than unlimited allowances; when reducing an allowance, set it to 0 first and confirm the transaction is mined before setting the new value; or use permit with tight deadlines. Users should understand that an old signed permit can still consume the nonce until it expires.

Status

Acknowledged (docs/holder-safety.md)

Informational Severity Issues

4. Irreversible Burns And Burn-Within-Allowance

Acknowledged

Path

`src/Monochrome.sol`

Function Name

`burn(uint256) and burnFrom(address,uint256) (inherited ERC20Burnable)`

Description

Burning permanently reduces totalSupply and cannot be reversed by anyone, including the team (there is no mint). In addition, burnFrom lets any address holding an allowance destroy the owner's tokens within that allowance, and a permit-created allowance carries the same power. An unlimited or long-lived approval therefore grants a spender the ability to irreversibly destroy the approved balance, not merely to transfer it.

Impact

A user who approves a malicious or compromised spender can have their MCR burned, with no recovery path. This is intended ERC-20 Burnable behavior and is described in docs/deployment.md, but the consequence differs from an ordinary transfer (the tokens leave supply entirely rather than moving to the spender).

Likelihood

Low; requires the holder to grant an allowance to an untrusted/compromised spender.

Recommendation

Keep allowances minimal, exact and short-lived; revoke approvals for spenders that no longer need them; treat permit signatures as equivalent to granting burn rights, not just transfer rights. Document this clearly for token holders.

Status

Acknowledged (README and docs/holder-safety.md)

Informational Severity Issues

5. Full Genesis Supply Concentrated In A Single Treasury

Acknowledged

Path

```
src/Monochrome.sol (constructor) / deployment configuration
```

Function Name

```
constructor(address initialHolder)
```

Description

100% of the 210,000,000 MCR supply is minted to one address in the constructor (on mainnet, the 3-of-4 Safe 0x839599c548FC1Afbe674c9b1691C78C9aCB3e391). There is no vesting, distribution or lockup in the token contract, and no mechanism to move the supply if the recipient loses access. The treasury Safe requires 3 of 4 owners; the loss of one key combined with one unavailable signer can immobilize the entire supply (a liveness risk), and three colluding owners can move the full supply (a custody concentration risk).

Impact

Not a defect in the token code. It is an economic/custody and operational risk: a single compromised or unavailable treasury can affect the entire supply. It does not allow the treasury to mint, inflate supply, alter the contract or seize other holders' balances; holders' balances remain theirs.

Likelihood

Low for the contract itself; the risk is operational (key management, Safe owner availability, governance of the treasury).

Recommendation

Follow the repository's operational guidance: independent key custody, periodic Safe-configuration re-verification, and a documented treasury/distribution policy. Consider a timelock or distribution contract for future allocations. These are operational controls outside the token contract.

Status

Acknowledged (docs/treasury-policy.md specifies custody, availability, verification and distribution records)

Functional Tests

Some of the tests performed are mentioned below:

- ✓ Metadata: `name() == "Monochrome"`, `symbol() == "MCR"`, `decimals() == 18`.
- ✓ Fixed supply: `totalSupply() == 210_000_000e18 == INITIAL_SUPPLY()`; minting to an arbitrary valid recipient; `totalSupply` cannot increase after construction.
- ✓ Constructor input: zero `initialHolder` reverts with MCR: zero `initialHolder`; any non-zero recipient receives the full supply and the deployer receives nothing.
- ✓ Transfers and allowances: balance moves, insufficient-balance/allowance reverts, zero-address receiver/spender rejection, `transferFrom` allowance consumption, infinite-allowance preservation, allowance rollback on failed calls.
- ✓ Burn: burn reduces supply and balance; cannot exceed balance; `burnFrom` respects finite allowance and infinite allowance; failed burns roll back supply and allowance.
- ✓ Permit (EIP-2612): gasless approval; exact-allowance setting; replay rejection; inclusive deadline; expired/nonce/signer/spender/value/owner/deadline/chain/contract/name/version substitution rejection; high-s and invalid-v rejection; zero-spender rollback; permit-created allowance supports `burnFrom`; domain separator and `eip712Domain()` introspection.
- ✓ Adversarial surface: no privileged selectors (`mint`, `owner`, `pause`, `upgradeTo`, role grants, etc. all revert); native ETH and unknown selectors rejected; tokens sent to the contract cannot be recovered.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates **everything the protocol does not fully control**.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ERC20 (OpenZeppelin v5.4.0)	Base Contract	Implements the ERC-20 ledger: transfer, transferFrom, approve, _mint, _burn, supply and balance accounting	Pinned, unmodified OpenZeppelin 5.4.0 at commit c64a1edb67b6e3f4a15cca8909c9482ad33a02b0 is correct and matches the reviewed runtime template	A dependency upgrade would change bytecode and invalidate the reviewed baseline; the token contract itself cannot change after deployment
ERC20Burnable (OpenZeppelin v5.4.0)	Base Contract	Adds burn(uint256) and burnFrom(address,uint256), routed through _burn	Correct allowance and balance checks; burn is irreversible	burnFrom lets an approved spender permanently destroy the owner's tokens within allowance (Informational)
ERC20Permit (OpenZeppelin v5.4.0)	Base Contract	Adds EIP-2612 permit, nonces, DOMAIN_SEPARATOR, eip712Domain; stores per-owner nonces	Correct EIP-712 domain and nonce handling; an EOA signs exactly the intended permit	Standard permit front-running / un-cancellable signed permits; ECDSA-only, so contract wallets (including the treasury Safe) are unsupported, and a Safe module cannot add ERC-1271 support to the deployed token
EIP712 / Nonces (OpenZeppelin v5.4.0)	Base Contract	Builds and caches the domain separator; tracks nonces per owner	Domain separator rebuilds on chain-ID change; nonce consumed exactly once	Cross-chain/contract/name/version replay is rejected; no residual risk identified

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ECDSA (OpenZeppelin v5.4.0)	Library	Recovers the permit signer and rejects high-s / invalid-v signatures	Signature malleability and invalid signatures are rejected	None identified; no ERC-1271 (contract-wallet) support
Solidity compiler	Toolchain	Compiles the pinned source at 0.8.28, optimizer 200, via_ir=false, evm_version=cancun	The pinned compiler has no applicable known-bug trigger for this source	A different compiler/settings produces different bytecode; recorded baseline must be re-verified
Ethereum chain environment	Network	Provides block.chainid (EIP-712 domain) and block.timestamp (permit deadline)	Chain ID is stable for the deployment; miners/validators order transactions	Timestamp is used only as a permit deadline; no economic decision depends on it
EOA permit signers	Off-chain actor	Sign EIP-2612 permit messages authorizing an allowance	The signer only signs messages they intend; the signature is not phishable into a different action	A mis-signed/unlimited permit grants allowance and burnFrom authority. The deadline limits only submission of the signature; once a permit is submitted the allowance persists until spent or changed, and does not expire at the deadline or because the nonce was consumed
Genesis treasury Safe	External contract	Holds 100% of the genesis supply (mainnet: 0x839599c548FC1Afb674c9b1691C78C9aCB3e391, 3-of-4)	Owner keys are independently held and the threshold is maintained	Key loss/unavailability can immobilize the supply; 3 colluding owners can move it (operational, not a token defect)
RPC / block explorer tooling	Off-chain infrastructure	Simulation, deployment, verification and read calls	Operator uses a correct chain, nonce and verified Safe configuration	Configuration errors at deployment; explicitly out of token-contract control

2. Entry & Exit Points Analysis (Function-Level)

This is the **core of the threat model**. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets **one table**.

Contract Name	Function	Category
Monochrome.sol	constructor(address)	What this function can do Mint the single fixed supply INITIAL_SUPPLY = 210_000_000e18 and assign the entire balance to initialHolder, emitting one Transfer(address(0), initialHolder, INITIAL_SUPPLY). What this function cannot/should not do Deploy with a zero recipient; mint more than once; be called again after deployment. It does not restrict the recipient: initialHolder may be any non-zero address, including the deployer, so a zero deployer allocation is a property of the recorded deployment, not a rule enforced by the contract. Main invariant(s) totalSupply() == INITIAL_SUPPLY == 210_000_000e18 and balanceOf(initialHolder) == INITIAL_SUPPLY at genesis. Access level Public, executes exactly once during deployment; zero recipient is rejected by require.

Contract Name	Function	Category
Monochrome.sol	name()	What this function can do Return the constant string "Monochrome". What this function cannot/should not do Mutate state or be changed after deployment. Main invariant(s) Metadata is immutable. Access level Public view, unrestricted.
Monochrome.sol	symbol()	What this function can do Return the constant string "MCR". What this function cannot/should not do Mutate state. Main invariant(s) Metadata is immutable. Access level Public view, unrestricted.
Monochrome.sol	decimals()	What this function can do Return 18. What this function cannot/should not do Change after deployment. Main invariant(s) decimals() == 18 for all MCR amounts. Access level Public view, unrestricted.

Contract Name	Function	Category
Monochrome.sol	totalSupply()	What this function can do Return the current supply (genesis supply minus all burns). What this function cannot/should not do Ever exceed INITIAL_SUPPLY; there is no mint path. Main invariant(s) $0 \leq \text{totalSupply()} \leq \text{INITIAL_SUPPLY}$; supply is conserved by transfers and reduced only by burn/burnFrom. Access level Public view, unrestricted.
Monochrome.sol	INITIAL_SUPPLY()	What this function can do Return the compile-time constant $210_000_000 * 10^{**18}$. What this function cannot/should not do Change after deployment. Main invariant(s) INITIAL_SUPPLY() is the upper bound of totalSupply() at all times. Access level Public constant getter, unrestricted.

Contract Name	Function	Category
Monochrome.sol	balanceOf(address)	What this function can do Return the stored balance of any address, including address(0) (always zero). What this function cannot/should not do Be manipulated except by transfer, transferFrom, burn, burnFrom and the genesis mint. Main invariant(s) Sum of all balances equals totalSupply(). Access level Public view, unrestricted.
Monochrome.sol	allowance(address,address)	What this function can do Return the remaining allowance for a spender. What this function cannot/should not do Be increased except by the owner's approve or a valid permit. Main invariant(s) Finite allowance is reduced by transferFrom/burnFrom; type(uint256).max is never decremented. Access level Public view, unrestricted.

Contract Name	Function	Category
Monochrome.sol	transfer(address,uint256)	What this function can do Move value from msg.sender to to, emitting Transfer. What this function cannot/should not do Move another account's balance; mint or burn; send to address(0); change total supply. Main invariant(s) totalSupply() is unchanged; sender balance decreases and receiver balance increases by value. Access level Public; caller may only spend its own balance.
Monochrome.sol	approve(address,uint256)	What this function can do Set allowance[msg.sender][spender] = value, emitting Approval; overwrite an existing allowance. What this function cannot/should not do Grant an allowance over another account's tokens; set a zero-address spender. Main invariant(s) Only the owner's allowance entry changes; supply is untouched. Access level Public; caller may only modify its own approval.

Contract Name	Function	Category
Monochrome.sol	transferFrom(address,address,uint256)	What this function can do Spend the caller's allowance from from, moving value to to and emitting Transfer; consume a finite allowance. What this function cannot/should not do Exceed the allowance or the owner's balance; send to address(0). It can spend an infinite allowance but does not decrement it. Main invariant(s) Allowance and balance are reduced atomically and rolled back on revert; supply is unchanged. Access level Public; requires allowance[from](msg.sender) >= value.

Contract Name	Function	Category
Monochrome.sol	burn(uint256)	What this function can do Permanently destroy value from msg.sender and reduce totalSupply, emitting Transfer(sender, address(0), value). What this function cannot/should not do Exceed the caller's balance; be reversed (no mint exists). Main invariant(s) totalSupply() + cumulativeBurned == INITIAL_SUPPLY; burn is irreversible. Access level Public; caller may only burn its own balance.
Monochrome.sol	burnFrom(address,uint256)	What this function can do Spend the caller's allowance from account, permanently destroy value, reduce supply and emit Transfer(account, address(0), value). What this function cannot/should not do Exceed the allowance or the account's balance; reverse the burn. Main invariant(s) Allowance and supply are reduced atomically; supply only decreases. Access level Public; requires allowance[account] [msg.sender] >= value.

Contract Name	Function	Category
Monochrome.sol	<pre>permit(address,address,uint256,uint256,uint8,bytes32,bytes32)</pre>	What this function can do Verify an EIP-712/EIP-2612 signature from owner, consume one nonce, set the allowance to value and emit Approval; any address may submit it. What this function cannot/should not do Authorize without a valid signature; be replayed (nonce); be reused across chains, contracts, names or versions (domain); set a zero-address spender; accept expired deadlines, invalid v, or high-s signatures. The deadline gates submission only: it does not make an already-established allowance expire. Main invariant(s) Each (owner, nonce) permits exactly one successful permit; the EIP-712 domain binds chain ID and verifying contract. An unused signed permit stays submittable until its deadline and is not cancelled by approve. Access level Public; effective authorization depends on a valid EOA signature.

Contract Name	Function	Category
Monochrome.sol	nonces (address)	What this function can do Return the next permit nonce for an owner. What this function cannot/should not do Be written except by a successful permit. Main invariant(s) The nonce increases by exactly one per successful permit and is unchanged on revert. Access level Public view, unrestricted.
Monochrome.sol	DOMAIN_SEPARATOR()	What this function can do Return the EIP-712 domain separator for the current chain ID and verifying contract. What this function cannot/should not do Be spoofed; it is derived from keccak256("EIP712Domain(string name,string version,uint256 chainId,address verifyingContract)"), name "Monochrome", version "1". Main invariant(s) A permit signed for one chain/contract cannot be replayed on another. Access level Public view, unrestricted.

Contract Name	Function	Category
Monochrome.sol	eip712Domain()	What this function can do Return the EIP-5267 domain fields: name "Monochrome", version "1", chainid, verifyingContract, zero salt, empty extensions. What this function cannot/should not do Be altered after deployment. Main invariant(s) Introspection matches DOMAIN_SEPARATOR(). Access level Public view, unrestricted.

3. Asset Flow Mapping (Critical Contracts)

This section identifies **where money actually lives** and **how it moves**.

3.1 Asset-Holding Contracts

List only contracts that **custody value**.

Contract	Asset Type	Custodied Assets
Monochrome.sol	ERC20 (MCR ledger)	No native ETH and no external-token custody. It is the authoritative ledger of MCR balances and allowances; its own contract address holds no tokens at genesis. Any MCR mistakenly transferred to address(Monochrome) is recorded as a normal balance and is permanently unrecoverable (no rescue function).
Genesis treasury Safe (external)	ERC20	Holds 100% of the 210,000,000 MCR genesis supply at deployment; controls it through ordinary Safe transactions. External to the token contract.

3.2 Asset Entry & Exit Functions

This table maps **money movement**, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Monochrome.sol	constructor	210,000,000 MCR (minted)	—	Deployer	One-time genesis mint; zero recipient rejected; must be deployed to the intended Safe. No second mint possible.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
Monochrome.sol	transfer	—	MCR from caller	Public	Sender-specific loss on a wrong/mistyped or unrecoverable destination; cannot affect other holders or supply.
Monochrome.sol	transferFrom	MCR from from (allowance)	MCR to to	Public spender	Spender can move up to allowance; finite allowance consumed atomically; standard allowance-replacement race (Informational).
Monochrome.sol	approve	—	Allowance (no asset movement)	Token owner	Overwrites existing allowance; unlimited approval enables later transfer and burn; cannot move funds by itself.
Monochrome.sol	permit	—	Allowance (no asset movement)	Any submitter on behalf of a signer	Signature must match owner/spender/value/nonce/deadline/domain; a leaked or mis-signed permit authorizes transfers and burns; outstanding permits are not cancelled by approve.
Monochrome.sol	burn	MCR from caller	— (supply destroyed)	Token holder	Irreversible; no mint can restore supply.
Monochrome.sol	burnFrom	MCR from account (allowance)	— (supply destroyed)	Public spender	Approved spender can irreversibly burn the owner's tokens within allowance; finite allowance consumed; permit-created allowances inherit this power.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of MCR Token. We performed our audit according to the procedure described above.

5 Informational issues were found and are Fixed/Acknowledged by the team.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses.

Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

September 2026

For

 **Monochrome**

 **QuillAudits**

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com